

Group Members

Maham Zafar 22-SE-11

Ayesha Latif 22-SE-29

Formal Methods in FIRE Alarm System

Project Description

This project models a **Fire Alarm System** using **formal methods** in the **B-Method** with Atelier B. The goal is to create a reliable and robust system that manages fire detection, alarm activation, and test modes. The machine ensures that the system behaves correctly in all scenarios by defining clear invariants and generating proof obligations for verification. The system is designed to transition between different states (OFF, ON, TEST) and handle operations like activating alarms, updating sensors, and entering/exiting test modes.

Machine Code

```
/* FireAlarmSystem
 * Author: MAHAM, AYESHA
 * Creation date: 11/25/2024
 */
MACHINE FireAlarmSystem

SETS
STATE = {OFF, ON, TEST} /* OFF: Alarm is inactive, ON: Alarm is
active, TEST: Alarm in test mode */

VARIABLES
alarm_state, sensor_status, test_mode_enabled

INVARIANT
alarm_state : STATE &
sensor_status : BOOL &
test_mode_enabled : BOOL &
(alarm_state = ON => sensor_status = TRUE) &
(alarm_state = TEST => test_mode_enabled = TRUE) &
(alarm_state /= TEST => test_mode_enabled = FALSE)
```

```

INITIALISATION
alarm_state := OFF || sensor_status := FALSE ||
test_mode_enabled := FALSE

OPERATIONS

/* Activate the alarm when fire is detected */
ActivateAlarm =
PRE
alarm_state = OFF & sensor_status = TRUE & test_mode_enabled =
FALSE
THEN
alarm_state := ON || test_mode_enabled := FALSE
END;

/* Deactivate the alarm manually */
DeactivateAlarm =
PRE
alarm_state = ON
THEN
alarm_state := OFF || sensor_status := FALSE ||
test_mode_enabled := FALSE
END;

/* Update the sensor status (fire detected or resolved) */
UpdateSensorStatus(new_status) =
PRE
new_status : BOOL
THEN
sensor_status := new_status ||
IF new_status = TRUE THEN
alarm_state := ON || test_mode_enabled := FALSE
ELSE
alarm_state := OFF || test_mode_enabled := FALSE
END
END;

/* Enter test mode for maintenance */
EnterTestMode =
PRE
alarm_state = OFF & test_mode_enabled = FALSE
THEN
alarm_state := TEST || test_mode_enabled := TRUE
END;

/* Exit test mode and return to normal state */

```

```

ExitTestMode =
PRE
alarm_state = TEST & test_mode_enabled = TRUE
THEN
alarm_state := OFF || test_mode_enabled := FALSE
END

END

```

	nPO	nPRi	nPRa	nUn	%Pr
Initialisation	2	0	2	0	100
Operation_ActivateAlarm	1	0	1	0	100
Operation_DeactivateAlarm	2	0	2	0	100
Operation_EnterTestMode	1	0	1	0	100
Operation_ExitTestMode	2	0	2	0	100
Operation_UpdateSensorStatus	3	0	3	0	100
FireAlarmSystem	11	0	11	0	100

Code Description

1. Purpose:

This machine models a simple **fire alarm system** with three states:

- OFF: The alarm is off.
- ON: The alarm is activated because a fire is detected.
- TEST: The system is in test mode for maintenance.

2. Components:

○ Variables:

- **alarm_state**: Tracks the current state of the alarm (OFF, ON, or TEST).
- **sensor_status**: A Boolean (TRUE or FALSE) that shows whether fire is detected.
- **test_mode_enabled**: A Boolean (TRUE or FALSE) indicating if the system is in test mode.

○ Invariant:

- Ensures variables remain in valid states.
- Example: If the alarm is ON, fire (**sensor_status**) must be detected. If the alarm is TEST, **test_mode_enabled** must be TRUE.

3. Operations:

- **ActivateAlarm**: Turns the alarm ON when fire is detected, but only if the system is not in test mode.
- **DeactivateAlarm**: Turns the alarm OFF manually when it is ON.
- **UpdateSensorStatus**: Updates whether fire is detected or not.
- **EnterTestMode**: Puts the system in test mode (TEST) for maintenance.
- **ExitTestMode**: Exits test mode and resets the system to OFF.

4. Initialization:

- Starts with the alarm OFF, no fire detected (**sensor_status** = FALSE), and test mode disabled (**test_mode_enabled** = FALSE).

Proof Obligations

Proof obligations are automatically generated by Atelier B to ensure the machine is **mathematically correct** and adheres to its invariants during all operations. These proofs verify that:

- The **invariant** is always preserved.
- Pre- and post-conditions for each operation are valid.

- Assignments within operations do not introduce inconsistencies.

In this system:

- **11 proof obligations** are generated.
- **11 out of 11 proofs** have been successfully completed.

The remaining proof obligation will require further refinement or analysis to ensure the system meets all correctness requirements. This demonstrates a high level of reliability in the current design.

Refinement Code

```
/* FireAlarmSystem_r
 * Author: MAHAM
 * Creation date: 12/13/2024
 */

REFINEMENT FireAlarmSystem_r
REFINES FireAlarmSystem

VARIABLES
alarm_state, sensor_status, test_mode_enabled

INITIALISATION
alarm_state := OFF || sensor_status := FALSE ||
test_mode_enabled := FALSE

OPERATIONS
ActivateAlarm =
PRE
alarm_state = OFF & sensor_status = TRUE & test_mode_enabled =
FALSE
THEN
alarm_state := ON || test_mode_enabled := FALSE
END;

/* Deactivate the alarm manually */
DeactivateAlarm =
PRE
alarm_state = ON
THEN
```

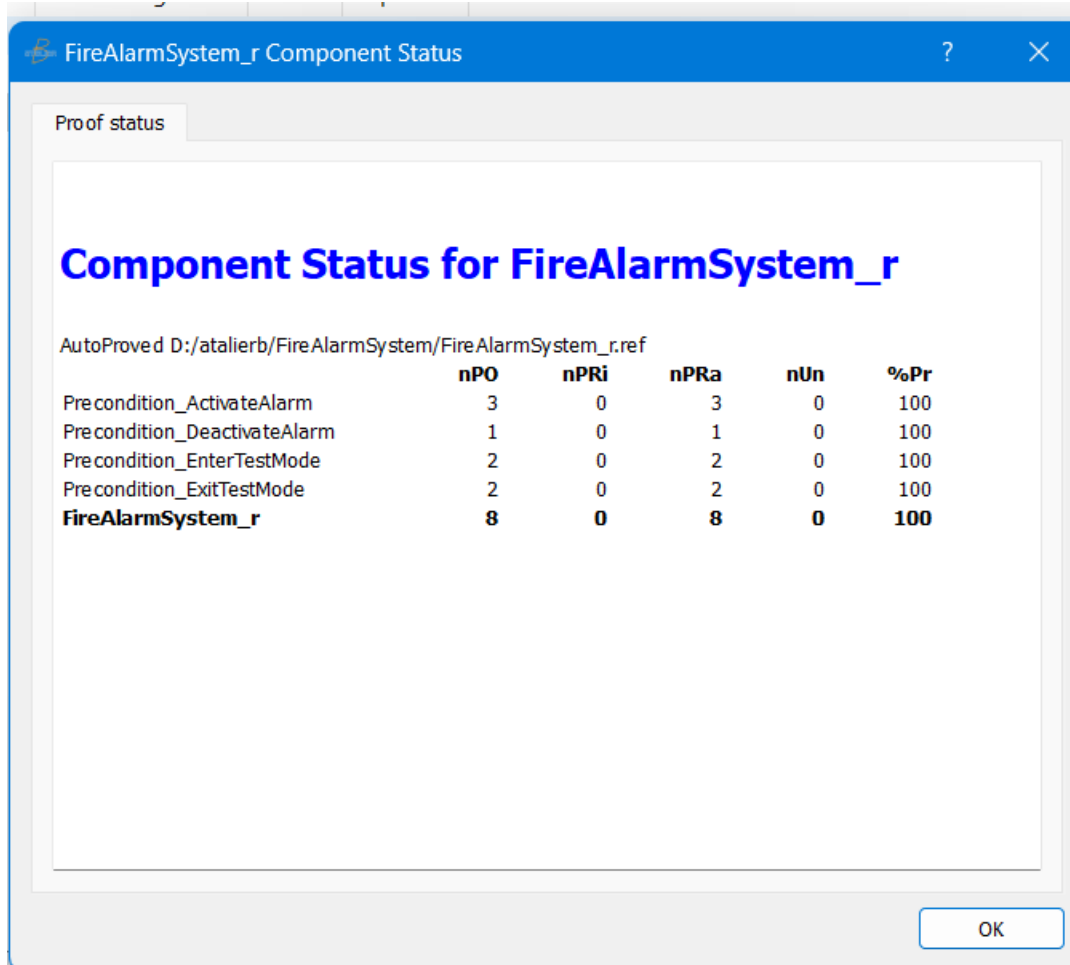
```
alarm_state := OFF || sensor_status := FALSE ||
test_mode_enabled := FALSE
END;

/* Update the sensor status (fire detected or resolved) */
UpdateSensorStatus(new_status) =
PRE
new_status : BOOL
THEN
sensor_status := new_status ||
IF new_status = TRUE THEN
alarm_state := ON || test_mode_enabled := FALSE
ELSE
alarm_state := OFF || test_mode_enabled := FALSE
END
END;

/* Enter test mode for maintenance */
EnterTestMode =
PRE
alarm_state = OFF & test_mode_enabled = FALSE
THEN
alarm_state := TEST || test_mode_enabled := TRUE
END;

/* Exit test mode and return to normal state */
ExitTestMode =
PRE
alarm_state = TEST & test_mode_enabled = TRUE
THEN
alarm_state := OFF || test_mode_enabled := FALSE
END

END
```



Implementation Code

```

/* FireAlarmSystem_i
* Author: Maham Zafar
* Creation date: 12/13/2024
*/

IMPLEMENTATION FireAlarmSystem_i
REFINES FireAlarmSystem_r
CONCRETE_VARIABLES
alarm_state, sensor_status, test_mode_enabled

INITIALISATION
alarm_state := OFF ; sensor_status := FALSE ; test_mode_enabled
:= FALSE

```

OPERATIONS

ActivateAlarm =

IF

alarm_state = OFF & sensor_status = TRUE & test_mode_enabled = FALSE

THEN

alarm_state := ON ; test_mode_enabled := FALSE

END;

/* Deactivate the alarm manually */

DeactivateAlarm =

IF

alarm_state = ON

THEN

alarm_state := OFF ; sensor_status := FALSE ; test_mode_enabled := FALSE

END;

/* Update the sensor status (fire detected or resolved) */

UpdateSensorStatus(new_status) =

IF

new_status = TRUE or new_status = FALSE

THEN

sensor_status := new_status ;

IF new_status = TRUE THEN

alarm_state := ON ; test_mode_enabled := FALSE

ELSE

alarm_state := OFF ; test_mode_enabled := FALSE

END

END;

/* Enter test mode for maintenance */

EnterTestMode =

IF

alarm_state = OFF & test_mode_enabled = FALSE

THEN

alarm_state := TEST ; test_mode_enabled := TRUE

END;

```

/* Exit test mode and return to normal state */
ExitTestMode =
IF
alarm_state = TEST & test_mode_enabled = TRUE
THEN
alarm_state := OFF ; test_mode_enabled := FALSE
END
END

```

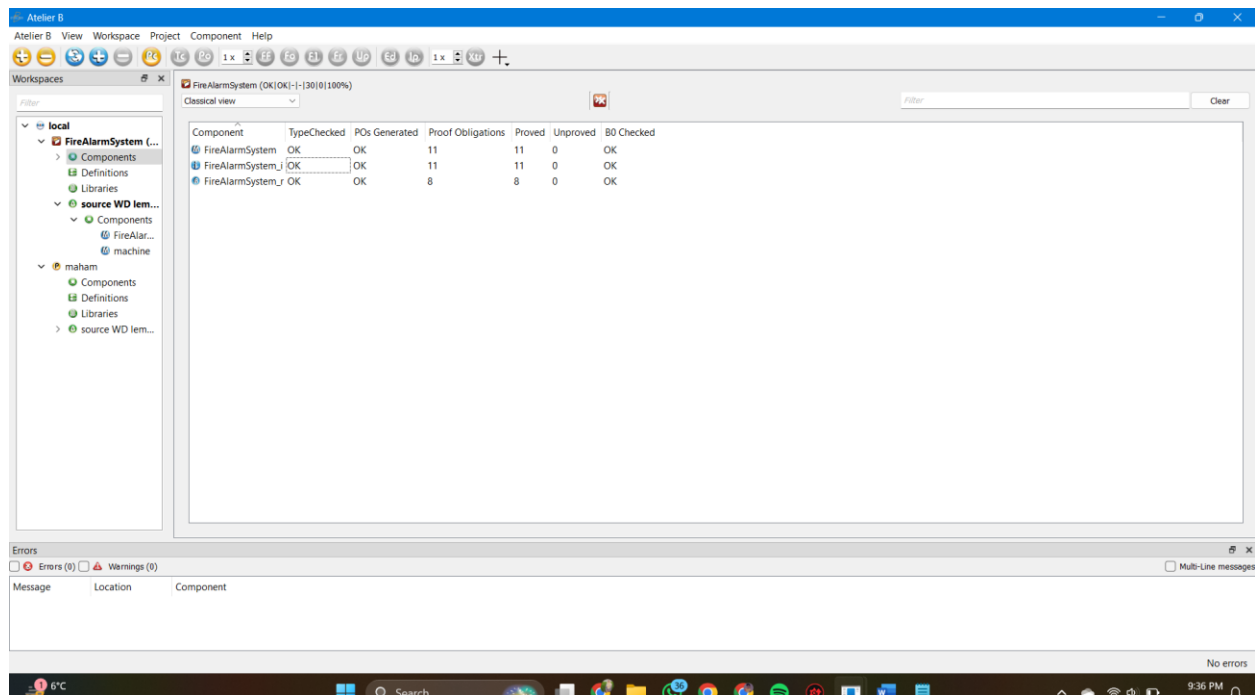
Proof status

Component Status for FireAlarmSystem_r

AutoProved D:/atalierb/FireAlarmSystem/FireAlarmSystem_r.ref

	nPO	nPRI	nPRa	nUn	%Pr
Precondition_ActivateAlarm	3	0	3	0	100
Precondition_DeactivateAlarm	1	0	1	0	100
Precondition_EnterTestMode	2	0	2	0	100
Precondition_ExitTestMode	2	0	2	0	100
FireAlarmSystem_r	8	0	8	0	100

OK



Generated Machine Code in C:

```
/* WARNING if type checker is not performed, translation could contain errors ! */
```

```
#include "FireAlarmSystem.h"
```

```
/* Clause CONCRETE_CONSTANTS */
```

```
/* Basic constants */
```

```
/* Array and record constants */
```

```
/* Clause CONCRETE_VARIABLES */
```

```
static FireAlarmSystem__STATE FireAlarmSystem__alarm_state;
```

```
static bool FireAlarmSystem__sensor_status;
```

```
static bool FireAlarmSystem__test_mode_enabled;
```

```
/* Clause INITIALISATION */
```

```
void FireAlarmSystem__INITIALISATION(void)
```

```
{
```

```
    FireAlarmSystem__alarm_state = FireAlarmSystem__OFF;
```

```
    FireAlarmSystem__sensor_status = false;
```

```
    FireAlarmSystem__test_mode_enabled = false;
```

```

}

/* Clause OPERATIONS */

void FireAlarmSystem__ActivateAlarm(void)
{
    if(((FireAlarmSystem__alarm_state == FireAlarmSystem__OFF) &&
        (FireAlarmSystem__sensor_status == true)) &&
        (FireAlarmSystem__test_mode_enabled == false))
    {
        FireAlarmSystem__alarm_state = FireAlarmSystem__ON;
        FireAlarmSystem__test_mode_enabled = false;
    }
}

void FireAlarmSystem__DeactivateAlarm(void)
{
    if(FireAlarmSystem__alarm_state == FireAlarmSystem__ON)
    {
        FireAlarmSystem__alarm_state = FireAlarmSystem__OFF;
        FireAlarmSystem__sensor_status = false;
        FireAlarmSystem__test_mode_enabled = false;
    }
}

void FireAlarmSystem__UpdateSensorStatus(bool new_status)
{
    if((new_status == true) ||
        (new_status == false))
    {
        FireAlarmSystem__sensor_status = new_status;
        if(new_status == true)
        {
            FireAlarmSystem__alarm_state = FireAlarmSystem__ON;
            FireAlarmSystem__test_mode_enabled = false;
        }
        else
        {
            FireAlarmSystem__alarm_state = FireAlarmSystem__OFF;
            FireAlarmSystem__test_mode_enabled = false;
        }
    }
}

```

```
void FireAlarmSystem__EnterTestMode(void)
{
    if((FireAlarmSystem__alarm_state == FireAlarmSystem__OFF) &&
        (FireAlarmSystem__test_mode_enabled == false))
    {
        FireAlarmSystem__alarm_state = FireAlarmSystem__TEST;
        FireAlarmSystem__test_mode_enabled = true;
    }
}
```

```
void FireAlarmSystem__ExitTestMode(void)
{
    if((FireAlarmSystem__alarm_state == FireAlarmSystem__TEST) &&
        (FireAlarmSystem__test_mode_enabled == true))
    {
        FireAlarmSystem__alarm_state = FireAlarmSystem__OFF;
        FireAlarmSystem__test_mode_enabled = false;
    }
}
```